Studio Nx

Backend(FastAPI/Python) 13 — 모델 관계 (project.members · project.tracks)

2026-10-07

Backend(FastAPI/Python) 13 — 모델 관계 (`project.members` · `project.tracks`)

항목	내용
파일	`studionx/backend/app/models/project.py` · `studionx/backend/app/models/track.py`
작업	P1-03 백엔드 공통 — ORM 모델 관계 (P1-03 마지막)
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	12 `Track` 모델 · 14 `check_password_rules()` (P1-04)

1. 하는 일

10 ~ 12 의 FK(`project_id`)를 **파이썬 속성**으로도 오갈 수 있게 한다. FK 는 DB 의 숫자 연결이고, 관계(`relationship`)는 그 연결을 파이썬 객체로 따라가게 해 준다.

관계가 없을 때	관계가 있을 때
`db.scalars(select(Track).where(Track.project_id == 12).order_by(Track.order_index))`	`project.tracks`
`db.get(Project, member.project_id)`	`member.project`
프로젝트를 `flush` 해서 번호를 받은 뒤 `ProjectMember(project_id=12, …)`	`project.members.append(ProjectMember(…))` — 번호는 저장할 때 자동으로 채워짐

속성	방향	값
`project.members`	프로젝트 → 멤버	`[ProjectMember, …]`
`project.tracks`	프로젝트 → 트랙	`[Track, …]` — `order_index` 순서
`member.project` · `track.project`	반대 방향	`Project`

2. 언제 쓰나

쓰는 곳	쓰는 방법
P1-13 프로젝트 만들기 (FR-122)	프로젝트에 OWNER 멤버 · MASTER 트랙을 붙여 `db.add(project)` 한 번 → 셋이 함께 저장
Phase 2 시퀀서	studioNx.html 을 열 때 `project.tracks` 로 트랙 목록을 화면 순서대로 읽음
Phase 6 협업	`project.members` 로 멤버 목록 · 권한 확인

```
# P1-13 create_project() 를 관계로 쓰면
project = Project(owner_id=user.user_id, title="My Song")
project.members.append(ProjectMember(user_id=user.user_id, role=MemberRole.OWNER))
project.tracks.append(Track(name="Master", track_type=TrackType.MASTER, order_index=0))
db.add(project)
db.commit()   # 하나라도 실패하면 셋 다 저장되지 않음
```

3. 코드

import (`project.py` 맨 위 — 바뀐 줄만)

```
from typing import TYPE_CHECKING, Any
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음 – 서로 불러오는 순환을 피함)
    from app.models.track import Track
```

import (`track.py` 맨 위 — 바뀐 줄만)

```
from typing import TYPE_CHECKING
from sqlalchemy.orm import Mapped, mapped_column, relationship

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음)
    from app.models.project import Project
```

두 파일 모두 **`relationship`** · **`TYPE_CHECKING`** 을 추가했다. `project.py` 는 `Track` 을, `track.py` 는 `Project` 를 타입 표시에만 쓴다.

함수 (`project.py` 의 `Project` 끝 · `ProjectMember` 끝)

```
class Project(Base):
    ...
    # 관계 – 행을 지우는 일은 DB 의 ON DELETE CASCADE 에 맡김 (passive_deletes)
    members: Mapped[list["ProjectMember"]] = relationship(
        back_populates="project", cascade="all, delete-orphan", passive_deletes=True
    )
    tracks: Mapped[list["Track"]] = relationship(
        back_populates="project",
        order_by="Track.order_index",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )


class ProjectMember(Base):
    ...
    project: Mapped[Project] = relationship(back_populates="members")
```

(`track.py` 의 `Track` 끝)

```
    project: Mapped["Project"] = relationship(back_populates="tracks")
```

문법 풀이

import 문법	뜻
`from typing import TYPE_CHECKING`	편집기 · 타입 검사기가 읽을 때만 `True`, 실제로 실행할 때는 `False` 인 값

import 문법	뜻
`if TYPE_CHECKING: from … import Track`	타입 표시에만 쓰는 import. `project.py` 와 `track.py` 가 서로를 실행 중에 불러오면 "아직 다 읽지 않은 파일" 오류(순환 import)가 날 수 있어 피함

식	뜻
`relationship()`	FK 를 따라 연결된 객체를 속성으로 줌. 어느 FK 를 쓸지는 SQLAlchemy 가 칸을 보고 찾음
`Mapped[list["ProjectMember"]]`	여러 개 (목록). `"…"` 글자로 적으면 아직 정의되지 않은 클래스도 가리킬 수 있음
`Mapped[Project]`	하나
`back_populates="project"`	반대쪽 속성 이름. 한쪽을 바꾸면 다른 쪽도 맞춰짐 (`member.project` ↔ `project.members`)
`order_by="Track.order_index"`	`project.tracks` 를 읽을 때 이 순서로 정렬
`cascade="all, delete-orphan"`	프로젝트를 `add` · `delete` 하면 자식도 함께. 목록에서 뺀 자식(고아)은 행도 지움
`passive_deletes=True`	프로젝트를 지울 때 자식 행을 파이썬이 하나씩 지우지 않고 DB 의 `ON DELETE CASCADE` 에 맡김

4. 흐름 (`db.add(project)` → `flush`)

1. `project.members` · `project.tracks` 에 붙은 객체도 함께 저장 대상이 된다 (`cascade`).
2. `INSERT INTO projects` → `project_id=12` 를 받는다.
3. 자식의 `project_id` 칸에 12 를 자동으로 넣고 `INSERT INTO project_members` · `INSERT INTO tracks`.
4. 하나라도 실패하면 `rollback` 으로 셋 다 없던 일이 된다.

5. 왜 이렇게

선택	이유
`members` 는 뼈대 TODO 그대로 (`cascade="all, delete-orphan"`)	멤버는 프로젝트 없이 존재할 수 없다
`tracks` 에도 `cascade` 를 추가 (뼈대 TODO 에는 `order_by` 만)	없으면 트랙을 읽은 상태로 프로젝트를 지울 때, SQLAlchemy 가 트랙의 `project_id` 를 비우려다 `NOT NULL` 오류를 낸다. 트랙도 프로젝트 없이 존재할 수 없다
`passive_deletes=True`	DB 에 이미 `ON DELETE CASCADE` 가 있다(11 · 12). 파이썬이 자식을 모두 읽어 하나씩 지우면 느리고 같은 일을 두 번 한다
보통 삭제는 소프트 삭제	P1-13 의 삭제는 `deleted_at` 기록이라 위 삭제 동작은 쓰이지 않는다. 관리자가 정말 지우는 경우를 위한 것
`User.projects` 는 아직 안 넣음	뼈대에 "필요해지면" 으로 되어 있다. 목록은 `ProjectMember` 로 JOIN 해서 찾는다 (P1-13 TODO)
`TYPE_CHECKING` 으로 import	두 파일이 서로를 가리키므로, 실행 중 import 순환을 피하면서 편집기 자동 완성은 살린다

6. 시험 (`tests/test_model_relationships.py`, 새로 만듦 · 6개)

실제 MySQL. 넣은 행은 끝나면 되돌린다 (시험 뒤 확인: 0행).

시험	확인한 것
프로젝트만 `add`	멤버 · 트랙도 저장되고, 자식의 `project_id` 가 자동으로 채워짐
반대 방향	`member.project is project` · `track.project is project`
트랙 순서	`order_index` 2 · 1 순서로 붙여도 다시 읽으면 `Master` · `Bass` · `Drums`
멤버를 목록에서 빼면	행이 지워짐 (`delete-orphan`)
자식을 읽은 상태로 프로젝트를 지우면	오류 없이 멤버 · 트랙 0개
트랙 하나가 실패하면 (`track_type` 빠짐)	`IntegrityError` → `rollback` 후 프로젝트도 없음 (FR-122)

전체 시험 76개 통과, `ruff` 검사 통과. 앱 시작 시 관계 설정(`configure_mappers()`)도 오류 없음.